

Spec-Driven Agents for RQE: **Specs, Tooling, and Trajectory-Based Evaluation**

Evaluation-led agent design on the Redis Query Engine

Srijith Rajamohan, Ph.D. · Yash Mandilwar · Chris Coleman · Itamar Haber · Adriano Amaral

Redis

THE PROBLEM

Reliability is the bottleneck between demo and production

Production means, every time:

- The right tools, in the correct order
- All the time

Answers must be correct, complete, and useful — ideally also efficient — and all of it measurable.

CASE STUDY

Troubleshooting the Redis Query Engine

An agent that diagnoses RQE (RedisSearch) issues: slow queries, indexing failures, memory pressure, stale results, vector search problems.

- Interprets raw diagnostics: FT.PROFILE, FT.INFO, SLOWLOG, shard metrics
- Staged workflow: Clarification → Diagnosis → Confirmation → Solution
- Must ground every conclusion in collected evidence

Why this domain is hard for agents

- Unlike SQL, parametric knowledge is often incorrect
- Very context dependent — needs to know what information is missing from a question
- Analysis may not catch all issues (completeness)
- Answers can be non-useful

OBSERVED FAILURE MODES

How agents actually go wrong

CORRECTNESS

Tool & file-read omission

Skips a required tool call, misses a file entirely, or reads it partially — the answer looks complete but is missing evidence.

Hallucination & misuse of information

Guesses paths or arguments that don't exist, forgets evidence already collected, or reasons illogically from what it has.

EFFICIENCY

Inefficient paths

Exploratory flailing: repeated reads of the same document, backtracking, dead-end searches.

None of these show up in an end-to-end "did it answer?" eval. They only show up in the trajectory.

APPROACH 1/2

Spec-driven design & tool granularity

Hierarchical specs + deliberately shaped tools — not a monolithic prompt:

Specs encode context & tool use

What context matters · which tools, how,
in what order · required inputs declared.

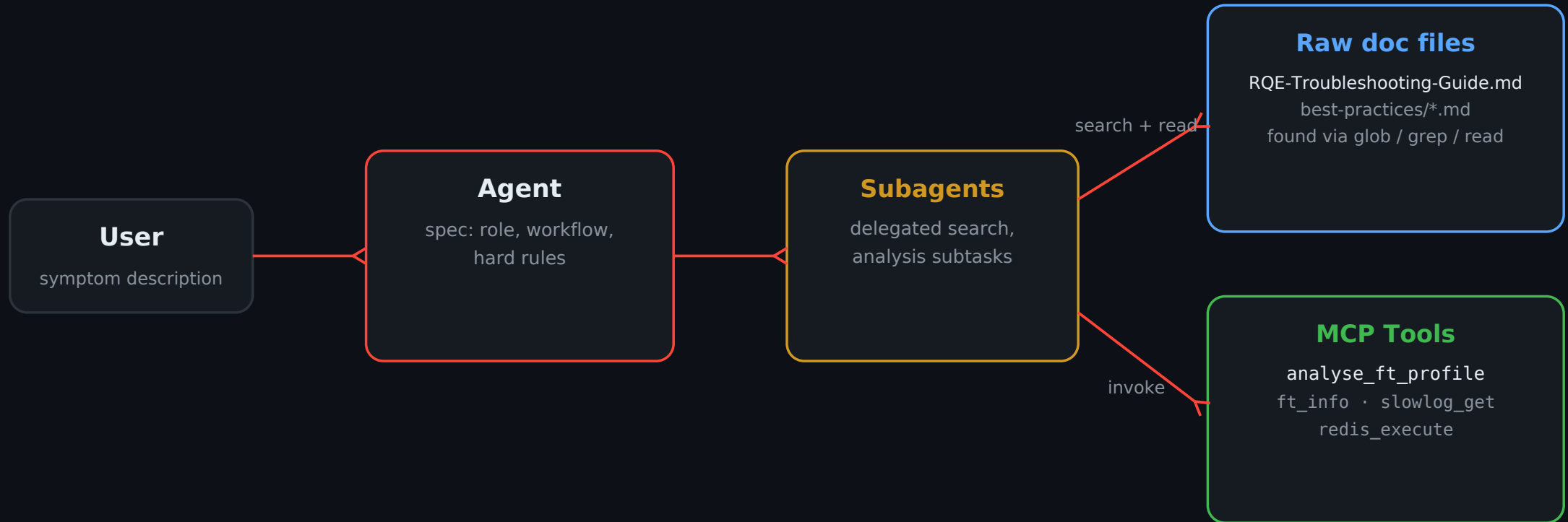
Right-sized tools

Transforms belong in tools, not the
context window: analyse_ft_profile →
compact summary + outliers.

Setup vs. discretionary tools

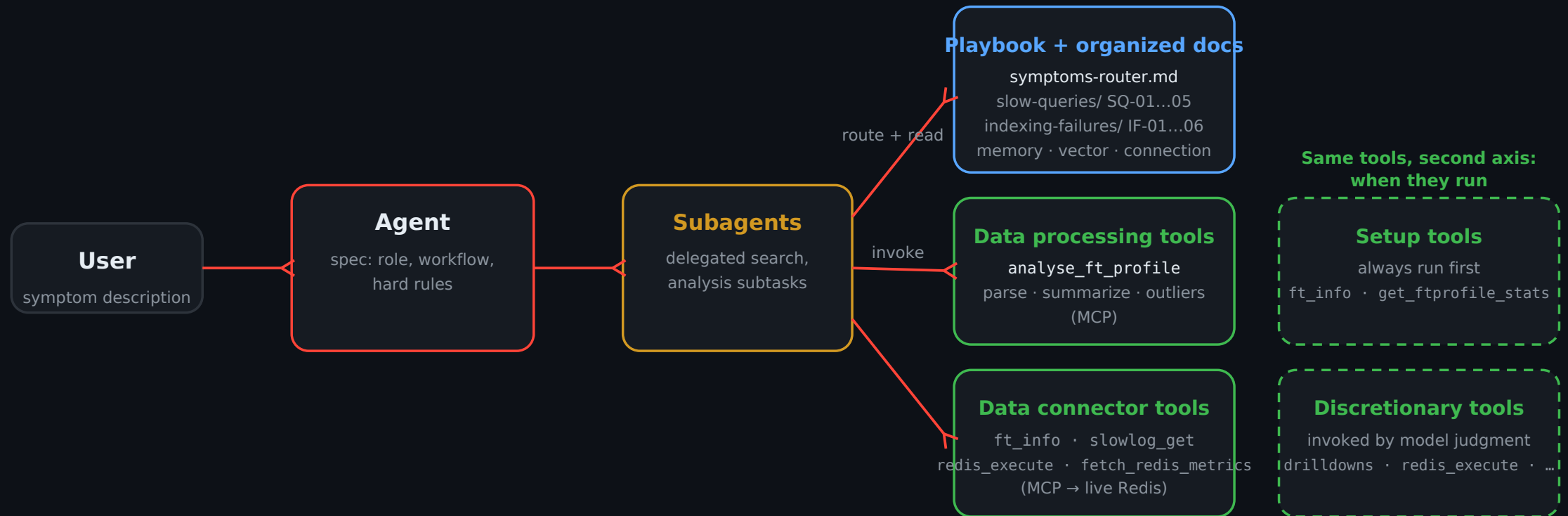
Setup: always run — ft_info before any
query. Discretionary: invoked by model
judgment as context requires.

A standard spec-driven agent



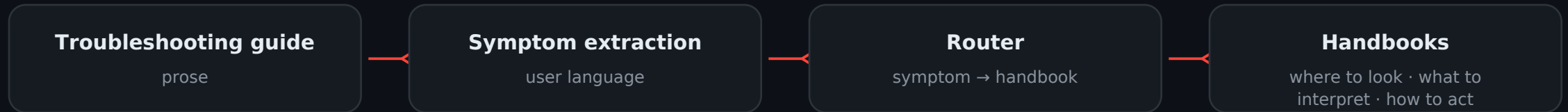
Knowledge is unstructured — the agent must find what it needs. This is the baseline we measure against.

Structured knowledge replaces raw docs



Two orthogonal tool abstractions: what they do — processing vs. connector — and when they run — setup (always, e.g. `ft_info`) vs. discretionary (model judgment).

Knowledge base → Agent Diagnostic Playbook



```
- symptom: "Any query with *word*  
  or big OR chains gets  
  dramatically slower."  
likely_problem_type:  
  "Wildcards and broad  
  text expansion"  
route_to_handbook:  
  - slow-queries/SQ-02-expensive-  
    wildcards-and-broad-text.md
```

- Symptoms phrased the way users complain
- Routing is a lookup, not a search
- Handbooks are self-contained: diagnostics, patterns, fixes

MEASURING IT

Trajectory-based evaluation

We instrument every session and score the full trajectory, not just the final answer:

Correctness

The answer is correct.

grounding · specificity · hallucinated paths · ordering (LCS similarity)

Completeness

The answer has identified all the issues.

tool/doc coverage · % sessions missing a required read · first-pass success

Efficiency

It has taken the most optimal path to the answer.

redundant/repeated reads · backtracking · dead-ends · tokens · duration · tool errors

Usefulness

The fix preserves the user's semantic intent.

e.g. "use fewer search terms" speeds the query but breaks what the user meant

Coverage makes omission visible; usefulness is the hardest to automate — it requires judging recommendations against intent, not just evidence.

EXPERIMENT 1 — TOOL GRANULARITY

Which MCP setup? Four designs, head-to-head

4 FT.PROFILE datasets × 5 sessions per setup (20 runs each):

Setup	Avg duration	Std dev	Tool calls	Quality
Baseline	50.8s	26.8s	217*	mixed
With-setup-tools	31.9s	5.6s	224	20/20
Minimal	31.6s	7.0s	251	20/20
Full	40.6s	13.3s	390	19/20

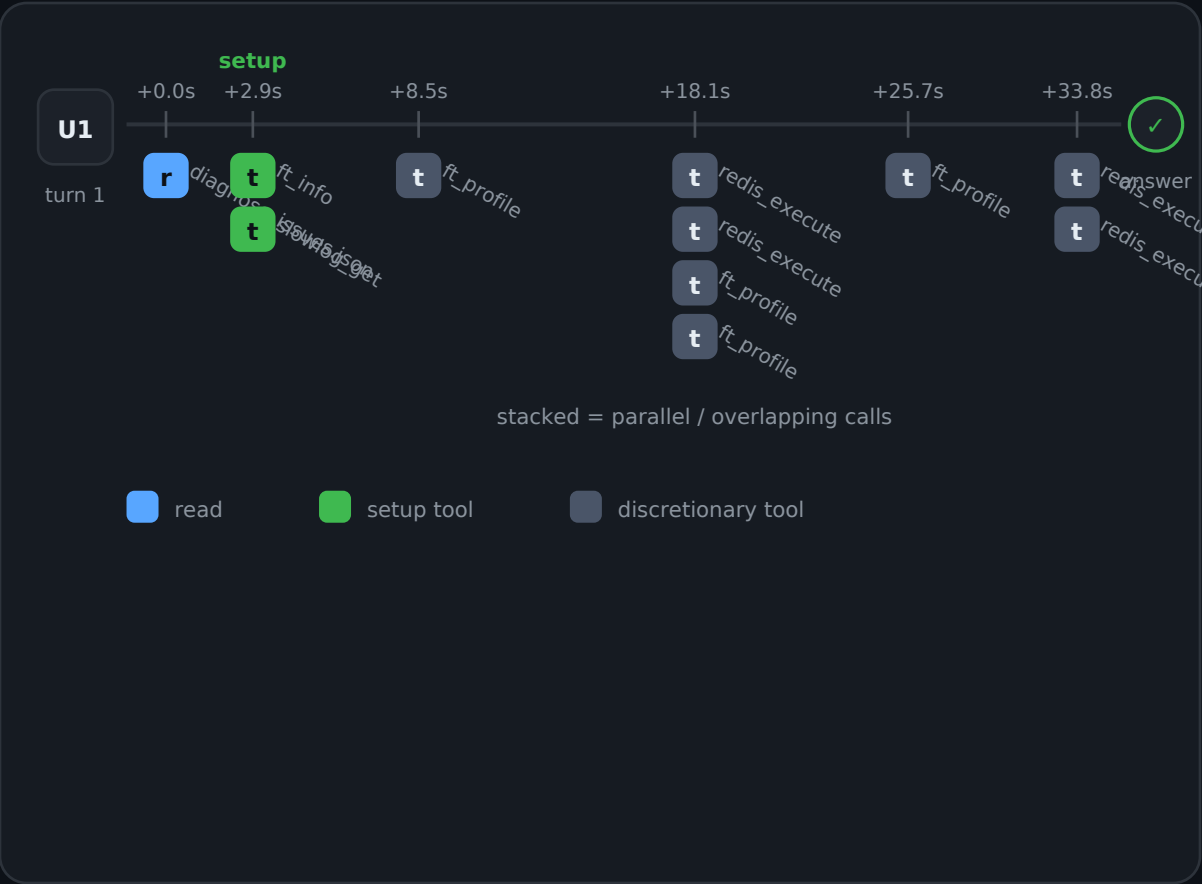
With-setup-tools wins 3/4 datasets: layered drilldowns — broad stats → diagnosis → iterator tree → details. Full is over-instrumented; the Baseline* leans on 49 bash + 5 grep fallbacks.

EXPERIMENT 1 — TOOL GRANULARITY

Granularity shapes trajectory consistency

	With-setup-tools	Minimal
Unique sequences / 20 runs	10	19
Avg LCS similarity	0.849	0.679
Avg tool-path length	11.2	12.6

LCS = longest common subsequence of tool calls, normalized — higher = more consistent ordering. With-setup-tools more consistent on every dataset.



One session, one turn — context read + setup tools first, then discretionary drilldowns → answer in ~34s.

EXPERIMENT 1 — TOOL GRANULARITY

Why each setup behaves the way it does

With-setup-tools — the only one with setup tools

Broad summary stats (timing, counts, iterators) establish runtime context before discretionary drilldowns (diagnosis, iterator tree, details). This layering induces a consistent investigation order.

Minimal — strong but unguided

Few tools, all necessary — so quality holds and it's marginally fastest. But no setup layer means each run improvises its own path: more calls, 19 unique trajectories in 20 runs.

Full — over-instrumented

Richest toolset, so the agent explores tools instead of the problem: 390 calls, slowest of the good setups, no quality gain to show for it.

Baseline — under-powered

Lacks targeted drilldowns, so the agent falls back to raw reads, grep, and bash (49 bash calls) — workable on simple cases, slow and inconsistent otherwise.

Granularity is a design dial: too few tools → fallback improvisation; too many → exploration overhead. Setup tools + discretionary drilldowns = just right — this is the demonstrated need for dynamic, runtime exploration.

EXPERIMENT 2 — KNOWLEDGE STRUCTURE

Raw guide vs. diagnostic playbook

Baseline: raw guide

Same agent spec, knowledge provided as a monolithic RQE-Troubleshooting-Guide.md plus best-practices docs. Agent must find what it needs.

Treatment: playbook

Same knowledge, restructured as symptoms-router + per-issue handbooks (SQ-04, SQ-07, SQ-11 relevant to the tasks).

- 5 sessions per condition, identical troubleshooting tasks over 5 FT.PROFILE inputs
- Both conditions call `analyse_ft_profile` 25× (5 per session) — tool access held constant
- Only the knowledge structure changes

EXPERIMENT 2 — RESULTS 1/2

Answer quality

Raw guide Playbook



Mean over 5 sessions per condition · grounding +45% rel., specificity +15% rel., dead-ends -52%

Cost & reliability

−43%

total tokens

326.5k → 185.0k

−48%

tool error rate

0.113 → 0.059

−15%

wall-clock duration

79.8s → 67.6s

Metric	Raw guide	Playbook
Cache-read tokens	218,573	108,442 (−50%)
Backtracking rate	0.015	0.000
Repeated doc reads / session	0.80	0.00
Unique documents covered	8.6	9.8
Required docs read in all 5 sessions	no (3/5 typical)	yes (5/5)

EVALUATION-LED DESIGN

The metrics also tell us what's still broken

- Hallucinated-path rate stays high in both (0.94 → 0.86): agents still guess near-miss paths (ft.profile vs ftProfile1.txt) before consulting the router — the next spec iteration targets this directly
- Grounding variance went up (σ 0.05 → 0.14) even as the mean improved: one or two playbook sessions skip an optional handbook — coverage tables point at exactly which reads were missed
- Exploration breadth/depth barely moved — the playbook doesn't make the agent less curious, it makes exploration land

This is the loop: trajectory metrics → identify failure mode → revise spec/playbook → re-measure. Design choices become testable hypotheses.

TAKEAWAYS

Three things to steal

1

Evaluation-led spec design

Let trajectory metrics — not intuition — drive how you structure agent specs, tool granularity, and ordering constraints.

2

KB → playbook pipelines

Restructure human knowledge bases into routed diagnostic playbooks: where to look, what to interpret, how to act — for efficient, reliable trajectories.

3

Measure the trajectory

Correctness, completeness, and usefulness of the answer are downstream of tool-call coverage, ordering, and duplication. Instrument those.

Same model, same tools, same knowledge — structure alone bought ~half the cost and a large step toward deterministic, debuggable trajectories.